

Day 3 — Databases, Models, ORM, Migrations, and Django Admin

Main goal

By the end of Day 3, you should understand:

- Why Python lists are not enough for real application data.
- What persistent data means.
- What a database is.
- What tables, rows, columns, and primary keys are.
- Why Django uses models.
- How a model maps to a database table.
- What model fields represent.
- What the ORM is.
- Why migrations exist.
- The difference between `makemigrations` and `migrate`.
- What Django admin is.
- How to create and inspect real `Post` records.
- How Django model instances are Python objects representing database rows.

By the end of the lesson, the blog will have a real `Post` model and actual posts stored in the database.

1. Review where the data came from on Day 2

On Day 2, the homepage used:

```
posts = [  
    {  
        "title": "My First Post",  
        "author": "Alice",  
    },  
    {  
        "title": "Learning Django",  
        "author": "Bob",  
    },  
]
```

Then the view passed that list into the template:

```
context = {  
    "posts": posts,  
}
```

The template displayed it:

```
{% for post in posts %}  
    <h2>{{ post.title }}</h2>  
    <p>{{ post.author }}</p>  
{% endfor %}
```

Question:

Where do these posts actually live?

Answer:

Inside Python code.

Question:

What happens if you want to add a new blog post while the application is running?

You would have to edit the Python file.

Question:

What happens if a user creates a blog post through the website?

There is nowhere permanent to store it yet.

This is the problem Day 3 solves.

2. What is persistence?

Introduce the idea of **temporary data** versus **persistent data**.

Temporary data exists only while a program is running or only inside the current Python code.

Example:

```
name = "Alice"
```

or:

```
posts = [  
    "Post 1",  
    "Post 2",  
]
```

This data exists in memory while the program runs.

If the program stops:

```
Program stops  
    ↓  
Memory is cleared  
    ↓  
Temporary data disappears
```

Persistent data is data that remains available even after the program stops and starts again.

Examples:

```
User accounts  
Blog posts  
Comments  
Orders  
Payments  
Products  
Messages
```

For our blog, we need:

```
Post created today  
    ↓  
Server stops  
    ↓  
Server starts tomorrow  
    ↓  
Post still exists
```

That requires persistent storage.

3. What is a database?

Begin with a simple definition:

A database is a system used to store, organize, and retrieve persistent data.

For our blog, we might eventually store:

Users
Posts
Comments
Categories

A database gives us a structured way to store that information.

Without a database:

Application
↓
Temporary Python values

With a database:

Application
↓
Database
↓
Persistent data

4. Relational database basics

Django commonly works with relational databases.

Examples include:

SQLite
PostgreSQL
MySQL
MariaDB

For this course, we will begin with SQLite.

Question:

Why SQLite?

Because it is simple and requires almost no setup.

Django can create a local database file for us.

For beginners, this lets us focus on understanding database concepts without first installing and configuring a separate database server.

5. Tables, rows, and columns

Use a blog example.

Imagine this database table:

`posts`

It might look like:

id	title	content	created_at
1	My First Post	Today I learned Django	...
2	Learning Templates	Templates are useful	...
3	Django Models	Models represent data	...

Explain each part.

Table

A table stores one type of thing.

For example:

`posts table`

stores posts.

Later:

`users table`

stores users.

Column

A column describes one piece of information each row may contain.

Example:

`title`
`content`

```
created_at
```

You can think of a column as similar to an attribute.

For a blog post:

```
Post
  title
  content
  created_at
```

Row

A row represents one stored record.

Example:

```
1 | My First Post | Today I learned Django | ...
```

That row represents one specific post.

6. Connect database structure to Python

You already know Python classes.

Example:

```
class Post:
    def __init__(self, title, content):
        self.title = title
        self.content = content
```

Then:

```
post = Post(
    "Learning Django",
    "Today I learned about models."
)
```

The object has:

```
post.title
post.content
```

Now compare that with a database table:

```
title          | content
-----
Learning Django | Today I learned...
```

Django models connect these two ideas.

7. What is a Django model?

A Django model is a Python class that describes the structure of data Django should store in the database.

Example:

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
```

This tells Django:

We want a type of stored data called `Post`.

Each `Post` should have:

```
title
content
```

Conceptually:

```
Django model
    ↓
describes
    ↓
database table
```

So:

```
class Post(models.Model):
```

eventually corresponds to something like:

```
blog_post
```

in the database.

8. Open `models.py`

Go to:

`blog/models.py`

It currently contains:

```
from django.db import models
```

Question:

Why does Django create this file automatically?

Because this is where the app's data models are normally defined.

Create the first model:

```
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
```

9. Understand `models.Model`

Focus on:

```
class Post(models.Model):
```

You already know inheritance from Python.

`Post` inherits from:

```
models.Model
```

That gives `Post` Django's model behavior.

Without Django:

```
class Post:
    ...
```

would just be an ordinary Python class.

With:

```
class Post(models.Model):
```

Django understands that:

Post represents stored database data

and gives it capabilities such as:

```
saving
querying
updating
deleting
validation helpers
database relationships
```

10. Understand model fields

Now examine:

```
title = models.CharField(max_length=200)
```

This is not a normal string value.

It is a model field declaration.

It tells Django:

This model has a field called title
and it stores short text.

Similarly:

```
content = models.TextField()
```

means:

This model has a field called content
and it stores longer text.

11. Common field types

Introduce only a few important ones.

CharField

```
title = models.CharField(max_length=200)
```

Used for relatively short strings.

Examples:

```
title
name
username
category name
```

TextField

```
content = models.TextField()
```

Used for larger amounts of text.

Examples:

```
blog content
description
article body
```

IntegerField

```
views = models.IntegerField()
```

Stores integers.

BooleanField

```
published = models.BooleanField(default=False)
```

Stores:

```
True
False
```

DateTimeField

```
created_at = models.DateTimeField(auto_now_add=True)
```

Stores a date and time.

We'll use this in the Post model.

12. Improve the Post model

Update:

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

Explain:

```
auto_now_add=True
```

means:

When this object is first created, automatically store the current date and time.

You do not have to manually provide it each time.

13. What about `id`?

Question:

We did not create an `id` field. How will each post be uniquely identified?

Django automatically gives a model a primary key if you do not define one yourself.

Conceptually, the table becomes:

```
id | title | content | created_at
```

So each post gets a unique identifier.

Example:

```
1
2
3
4
```

14. What is a primary key?

A primary key is a value that uniquely identifies one row.

Example:

```
id | title
-----
1  | My First Post
2  | My First Post
```

The titles are identical.

But:

```
id = 1
```

and:

```
id = 2
```

identify different rows.

This matters because titles are not guaranteed to be unique.

Question:

If two posts have the same title, how can the database distinguish them?

By their primary key.

Later we will use URLs such as:

```
/posts/1/
/posts/2/
```

where the number identifies the post.

15. Important clarification: defining a model does not immediately change the database

At this point, we have written:

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

Question:

Does the database already have a posts table?

No.

We have only changed Python code.

This introduces migrations.

16. What is a migration?

A migration describes a database structure change.

For example, we are saying:

Create a table for Post

with fields:

```
title
content
created_at
```

Django needs a controlled way to turn model changes into database changes.

Conceptually:

```
models.py changed
    ↓
migration created
    ↓
migration applied
    ↓
database changed
```

17. Why not just edit the database directly?

Imagine the project starts with:

```
Post:
title
content
```

A week later, you add:

```
created_at
```

Then later:

```
author
```

Then:

```
published
```

You need a record of how the database structure changed over time.

Migrations give Django that history.

Think of them as:

Version-controlled instructions for changing database structure.

18. makemigrations

Run:

```
uv run python manage.py makemigrations
```

Expected output will be similar to:

```
Migrations for 'blog':
  blog/migrations/0001_initial.py
    + Create model Post
```

Explain:

`makemigrations` looks at changes in model definitions and creates migration files describing those changes.

It does not yet necessarily apply the change to the database.

Mental model:

```
models.py
  ↓
makemigrations
  ↓
migration file
```

19. Inspect the migration folder

You now have something similar to:

```
blog/
├── migrations/
│   ├── __init__.py
│   └── 0001_initial.py
```

Explain:

0001

means this is the first migration for this app.

Do not teach the generated migration code in depth.

Simply show:

Django generated instructions representing the model we created.

The important thing is understanding why the file exists.

20. migrate

Now run:

```
uv run python manage.py migrate
```

Explain:

`migrate` applies migration instructions to the actual database.

Mental model:

```
migration files
  ↓
migrate
```

↓
database structure changes

So:

`makemigrations`

means:

Prepare database-change instructions.

And:

`migrate`

means:

Apply those instructions.

This distinction must be very clear.

21. `makemigrations` VS `migrate`

Use this exact comparison.

`makemigrations`

Reads model changes and creates migration files.

Python model changes
↓
migration instructions

`migrate`

Applies migration files to the database.

migration instructions
↓
database changes

Question:

If you add a new field to a model but never run `makemigrations`, has Django recorded the database change?

No.

Question:

If you run `makemigrations` but never run `migrate`, has the actual database structure necessarily changed?

No.

22. Where is the database?

After migration, look at the project root.

You should see:

```
db.sqlite3
```

Explain:

This file is our SQLite database.

So now:

```
beginner-blog/  
├── db.sqlite3  
├── manage.py  
├── blog/  
├── config/  
└── ...
```

The database is not Python code.

It is a separate data store used by the application.

23. What is the ORM?

Now introduce one of the most important Django concepts.

ORM means:

Object-Relational Mapping

Break the name apart.

Object

Python object.

Example:

```
post.title
```

Relational

Relational database.

Example:

```
posts table
```

Mapping

Django connects the two.

Conceptually:

```
Python object
    ↓
Django ORM
    ↓
database row
```

24. Why is the ORM useful?

Without an ORM, you might write SQL directly.

Example conceptually:

```
SELECT * FROM blog_post;
```

With Django ORM:

```
Post.objects.all()
```

Instead of:

```
INSERT INTO blog_post ...
```

you can work with:

```
Post.objects.create(...)
```

Important explanation:

Django's ORM does not mean the database stops using SQL.

Django translates your Python ORM operations into database operations.

You write:

```
Post.objects.all()
```

Django communicates with the database appropriately.

25. Think of models as Python interfaces to database data

A row:

```
id: 1
title: Learning Django
content: Models are useful
```

can be represented in Python as a model instance.

Conceptually:

```
post.id
post.title
post.content
```

This is why templates can later use:

```
{{ post.title }}
```

The template is working with a Django model object.

26. Open the Django shell

We can experiment without building a web form yet.

Run:

```
uv run python manage.py shell
```

Now import the model:

```
from blog.models import Post
```

Question:

What is `Post` here?

The model class we defined.

27. Create a post through the ORM

Run:

```
post = Post(  
    title="My First Database Post",  
    content="This post is stored in the database."  
)
```

At this moment, you have created a Python model object.

Question:

Has it definitely been saved to the database yet?

Not yet.

Run:

```
post.save()
```

Now the object is stored.

Inspect:

```
post.id
```

You should get something like:

```
1
```

This id was assigned by the database/Django model system.

28. Another way to create an object

You can also write:

```
Post.objects.create(  
    title="Learning the ORM",  
    content="Django lets us work with database records using Python."  
)
```

This creates and saves the object in one step.

Compare:

```
post = Post(...)  
post.save()
```

with:

```
Post.objects.create(...)
```

Both are valid.

29. Retrieve all posts

Run:

```
Post.objects.all()
```

This returns something like:

```
<QuerySet [...]>
```

Introduce the word **QuerySet** lightly.

A QuerySet represents a collection of database records retrieved through Django's ORM.

For now:

```
Post.objects.all()
```

means:

Give me all Post records.

We will study QuerySets properly on Day 4.

30. Retrieve one object

Try:

```
Post.objects.get(id=1)
```

Store it:

```
post = Post.objects.get(id=1)
```

Then:

```
post.title
```

and:

```
post.content
```

This shows the relationship clearly:

```
database row
    ↓
Django model instance
    ↓
Python object
```

31. Why does the shell output look ugly?

You may see something like:

```
<Post: Post object (1)>
```

That is not very useful.

Improve the model.

In:

```
blog/models.py
```

add:

```
def __str__(self):  
    return self.title
```

Full model:

```
from django.db import models  
  
class Post(models.Model):  
    title = models.CharField(max_length=200)  
    content = models.TextField()  
    created_at = models.DateTimeField(auto_now_add=True)  
  
    def __str__(self):  
        return self.title
```

Explain:

`__str__` is a normal Python special method.

It tells Python/Django how to represent this object as a readable string.

Without it:

```
Post object (1)
```

With it:

```
My First Database Post
```

This is not database magic.

It is normal Python object behavior being used by Django.

32. Does changing `__str__` require a migration?

Question:

We changed the model class. Should we run `makemigrations` again?

No.

Why?

Because:

```
def __str__(self):
```

does not change the database structure.

It only changes Python behavior.

This is an important distinction.

Changes such as:

```
title = models.CharField(...)
```

may affect the database.

Changes such as:

```
def __str__(self):
```

do not.

33. Introduce Django admin

Question:

Do we have to use the Python shell every time we want to create a post?

No.

Django includes an administrative interface.

This is called **Django admin**.

It lets authorized administrative users manage application data through a web interface.

Examples:

```
Create records  
View records  
Edit records  
Delete records  
Manage users
```

Important clarification:

Django admin is not normally the public interface for normal users.

It is primarily a management interface.

Later our normal users will create posts through our own blog pages.

34. Register the Post model in admin

Open:

```
blog/admin.py
```

It currently contains:

```
from django.contrib import admin
```

Update:

```
from django.contrib import admin
```

```
from .models import Post
```

```
admin.site.register(Post)
```

Explain:

Defining a model does not automatically make it appear in Django admin.

You have to register it.

35. Create an admin user

Run:

```
uv run python manage.py createsuperuser
```

Django will prompt for information such as:

```
Username
Email
Password
```

Explain the idea of a superuser:

A superuser is an administrative account with broad permissions inside Django admin.

Do not use normal user accounts for this role casually in a real system.

36. Open the admin

Run the server if it is not already running:

```
uv run python manage.py runserver
```

Visit:

```
http://127.0.0.1:8000/admin/
```

Log in.

You should now see:

```
Authentication and Authorization
```

```
Users
Groups
```

```
Blog
```

```
Posts
```

The `Post` model appears because it was registered.

37. Create a real blog post through admin

Create:

Title:
Learning Django Models

Content:
Today I learned how Django models represent database data.

Save it.

Create another:

Title:
Understanding Migrations

Content:
Migrations help Django keep the database structure synchronized with model changes.

Now these posts exist in:

`db.sqlite3`

They are no longer hard-coded Python dictionaries.

38. Trace what happened when creating a post

Conceptually:

```
Admin form
  ↓
Django receives submitted data
  ↓
Post model instance created
  ↓
ORM
  ↓
Database
  ↓
row stored
```

The important connection is:

HTML form
↓
Django
↓
Python model object
↓
database row

Later, when we build our own Create Post page, the same basic idea will happen outside admin.

39. Connect the admin record to the shell

Create a post through admin.

Then open:

```
uv run python manage.py shell
```

Run:

```
from blog.models import Post
```

Then:

```
Post.objects.all()
```

The post created in admin appears.

Question:

Why can the shell see a post created through the admin page?

Because both are accessing the same database.

This reinforces that data lives in the database, not inside the admin page.

40. Replace the temporary Python list

Now return to:

```
blog/views.py
```

Day 2 had:

```
posts = [  
    {  
        "title": "My First Post",  
        "author": "Alice",  
    },  
    ...  
]
```

Delete that temporary list.

Import the model:

```
from .models import Post
```

Then:

```
def home(request):  
    posts = Post.objects.all()  
  
    context = {  
        "posts": posts,  
    }  
  
    return render(request, "blog/home.html", context)
```

Now the source of data has changed.

Before:

```
Python list  
  ↓  
view  
  ↓  
template
```

Now:

```
Database  
  ↓  
ORM  
  ↓  
Post model objects  
  ↓  
view  
  ↓  
template
```

This is one of the most important transitions in the course.

41. Update the template

The current template may use:

```
{% for post in posts %}

    <article>

        <h3>{{ post.title }}</h3>

        <p>{{ post.author }}</p>

    </article>

{% endfor %}
```

We have not added `author` yet.

For now, use:

```
{% for post in posts %}

    <article>

        <h3>{{ post.title }}</h3>

        <p>{{ post.content }}</p>

        <p>
            Created: {{ post.created_at }}
        </p>

    </article>

{% endfor %}
```

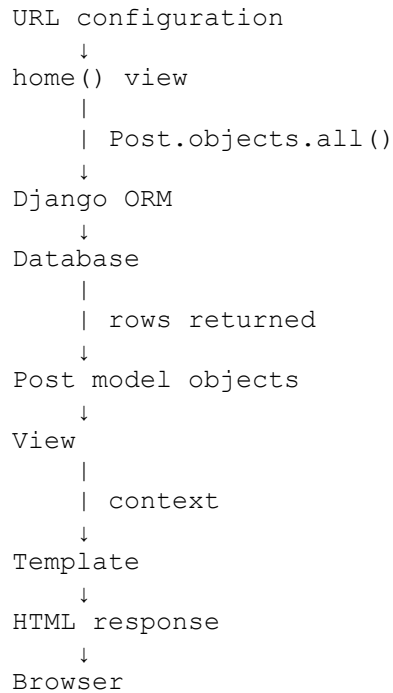
Now refresh the homepage.

The posts shown on the page are coming from the database.

42. Trace the full Day 3 request flow

This expands the Day 2 mental model.

```
Browser
  |
  | GET /
  ↓
```



This is the central architecture for Day 3.

43. Important concept: model class vs model instance

This distinction must be clear.

Model class

Post

The model definition.

It describes what a Post is.

Example:

```
class Post(models.Model):  
    ...
```

Model instance

```
post
```

One actual Post object.

Example:

```
post = Post.objects.get(id=1)
```

Think:

```
Post
```

means:

The type of object.

```
post
```

means:

One specific object of that type.

Same idea as normal Python:

```
class Car:  
    ...
```

and:

```
car = Car()
```

44. Model instance vs database row

Suppose the database contains:

```
id | title  
-----  
1  | Learning Django
```

Django can represent that row as a Python object:

```
post = Post.objects.get(id=1)
```

Then:

```
post.id
```

returns:

```
1
```

and:

```
post.title
```

returns:

```
Learning Django
```

So:

```
database row
```

and:

```
Python model instance
```

represent the same application data at different layers.

45. Modify a record in the shell

Retrieve:

```
post = Post.objects.get(id=1)
```

Change:

```
post.title = "My Updated Django Post"
```

Question:

Has the database definitely changed yet?

Not yet.

Save:

```
post.save()
```

Now refresh the admin or homepage.

The title changes.

This demonstrates:

```
Python object change
    ↓
save()
    ↓
database update
```

46. Delete a record in the shell

Retrieve:

```
post = Post.objects.get(id=1)
```

Then:

```
post.delete()
```

Now:

```
Post.objects.all()
```

will no longer include it.

This gives a preview of CRUD.

```
Create
Read
Update
Delete
```

We are not building public CRUD pages yet, but the underlying database operations already exist.

47. Connect today's work to CRUD

We have already performed:

Create

```
Post.objects.create(...)
```

Read

```
Post.objects.all()
```

and:

```
Post.objects.get(...)
```

Update

```
post.title = "..."  
post.save()
```

Delete

```
post.delete()
```

This demonstrates that CRUD is not a Django page feature.

CRUD describes common operations performed on stored data.

Later we will build website pages that trigger these operations.

48. Common beginner confusion: model vs database

Question:

Is the model itself the database?

No.

The model is Python code.

Example:

```
class Post(models.Model):
```

The database is the persistent data store.

Example:

```
db.sqlite3
```

Relationship:

```
Model definition
  ↓
Django ORM
  ↓
Database
```

49. Common beginner confusion: migration vs data

Question:

Does a migration contain all the blog posts?

No.

Migration files describe changes to database structure.

Example:

```
Create Post table
Add published field
Add author field
```

Actual blog posts are data stored inside the database.

So:

migration

is primarily about **schema/structure**.

database rows

are the actual application data.

50. Introduce the word schema

A database **schema** describes the structure of the database.

For our Post table, the schema conceptually says:

Post

```
id
title
content
created_at
```

The actual data might be:

```
1 | Learning Django | ... | ...
2 | Models          | ... | ...
```

So:

```
schema = structure
data = actual stored values
```

Migrations change the schema.

The admin creates and changes data.

51. Add another field to demonstrate migrations

Now deliberately change the model:

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

Explain:

```
published = models.BooleanField(default=False)
```

means every post now has:

```
published = True
```

or:

```
published = False
```

Existing rows did not previously have this column.

Question:

What must happen now?

Run:

```
uv run python manage.py makemigrations
```

Then:

```
uv run python manage.py migrate
```

Now the schema includes:

```
published
```

This is much more useful than explaining migrations only once.

52. Why is `default=False` useful?

Existing posts already exist in the database.

When adding a new field:

```
published = models.BooleanField(...)
```

Django needs to know what value existing rows should receive.

By writing:

```
default=False
```

we tell Django:

Existing and newly created posts should default to unpublished unless specified otherwise.

53. See the new field in admin

Refresh admin.

Edit a post.

You should now see a checkbox for:

Published

This demonstrates:

```
Model changed
  ↓
migration
  ↓
database changed
  ↓
admin automatically reflects model structure
```

This is one of Django's strengths.

54. Optional admin improvement

You can improve the admin display:

```
from django.contrib import admin

from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = (
        "title",
        "published",
        "created_at",
    )
```

Now the Post list in admin can show:

```
Title
Published
Created at
```

Explain only at a high level:

`ModelAdmin` lets us customize how a model appears in Django admin.

Do not go deeply into admin customization yet.

55. Complete `Post` model for Day 3

At the end of Day 3:

```
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

Do not add `author` yet.

That relationship is better introduced when authentication and ownership are taught.

56. Complete homepage view

```
from django.shortcuts import render

from .models import Post

def home(request):
    posts = Post.objects.all()

    context = {
        "posts": posts,
    }

    return render(request, "blog/home.html", context)

def about(request):
    return render(request, "blog/about.html")

def contact(request):
    return render(request, "blog/contact.html")
```

57. Complete homepage template

```
{% extends "blog/base.html" %}

{% block title %}
    Home | BeginnerBlog
{% endblock %}

{% block content %}

    <h1>BeginnerBlog</h1>

    <h2>Latest Posts</h2>

    {% if posts %}

        {% for post in posts %}

            <article>

                <h3>
                    {{ post.title }}
                </h3>

                <p>
                    {{ post.content }}
                </p>

                <p>
                    Created: {{ post.created_at }}
                </p>

                {% if post.published %}

                    <p>Published</p>

                {% else %}

                    <p>Draft</p>

                {% endif %}

            </article>

        {% endfor %}

    {% else %}

        <p>No posts are available yet.</p>

    {% endif %}
```

```
{% endblock %}
```

58. Day 3 vocabulary

Persistence

The ability for data to remain stored after the program stops.

Database

A system used to store and retrieve persistent data.

Relational database

A database that organizes data into related tables.

Table

A structure containing records of one type.

Row

One individual record in a table.

Column

One type of value stored for each record.

Primary key

A value that uniquely identifies a database row.

Schema

The structure of the database.

Model

A Django Python class describing application data and how it maps to the database.

Model field

A model attribute describing a piece of stored data.

Model instance

One Python object representing one model record.

ORM

Object-Relational Mapping.

The system Django uses to let Python objects interact with relational database data.

Migration

A recorded instruction describing a database schema change.

makemigrations

Creates migration files based on model changes.

migrate

Applies migrations to the database.

QuerySet

A Django object representing a collection of database query results.

Django admin

Django's built-in administrative interface for managing application data.

59. Day 3 concept questions

Question 1

Why was the Python list from Day 2 not suitable for real blog posts?

Because the data was hard-coded and not persistently stored.

Question 2

What does a database solve?

It gives the application persistent storage.

Question 3

What is the difference between a row and a column?

A row represents one record.

A column represents one type of value stored for records.

Question 4

Why does a row need an id?

The id uniquely identifies that row.

Question 5

Is a Django model the database?

No.

A model is Python code describing and interacting with database data.

Question 6

What does the ORM do?

It maps between Python objects and relational database data.

Question 7

What is the difference between `makemigrations` and `migrate`?

`makemigrations` creates migration instructions.

`migrate` applies those instructions to the database.

Question 8

What does `Post.objects.all()` mean?

Retrieve all Post records.

Question 9

What is the difference between `Post` and `post`?

`Post` is the model class.

`post` usually refers to one model instance.

Question 10

Does adding `__str__()` require a migration?

No.

It changes Python behavior, not database structure.

Question 11

Does adding a new model field require a migration?

Usually yes, because it changes the database schema.

Question 12

Does Django admin store data separately?

No.

It uses the same database as the rest of the Django application.

60. Day 3 exercise 1 — Create records using admin

Create at least three posts through Django admin.

Each post should contain:

```
title
content
published state
```

Then verify they appear on the homepage.

The goal is to understand:

```
Admin
  ↓
database
  ↓
view
  ↓
template
```

61. Day 3 exercise 2 — Use the Django shell

Open:

```
uv run python manage.py shell
```

Import:

```
from blog.models import Post
```

Perform:

```
Post.objects.all()
```

Create a record.

Retrieve one record.

Update its title.

Save it.

Delete it.

This gives you direct experience with database CRUD through the ORM.

62. Day 3 exercise 3 — Add a new field

Add:

```
summary = models.CharField(  
    max_length=300,  
    blank=True  
)
```

Then decide:

What commands must be run?

Expected:

```
uv run python manage.py makemigrations  
uv run python manage.py migrate
```

Then edit a post through admin and add a summary.

Display the summary on the homepage.

63. Explain `blank=True` simply

For now:

```
blank=True
```

means:

Django allows this field to be left empty during validation.

Do not introduce `null=True` deeply yet.

That distinction can come later.

64. Day 3 debugging exercise 1

Suppose you add:

```
summary = models.CharField(max_length=300)
```

but refresh admin and the field does not appear properly in the database-backed form.

Question:

What might you have forgotten?

Likely:

```
makemigrations  
migrate
```

65. Day 3 debugging exercise 2

Suppose:

```
Post.objects.all()
```

produces an error.

Question:

What is wrong?

Correct:

```
Post.objects.all()
```

It is:

```
objects
```

plural.

66. Day 3 debugging exercise 3

Suppose:

```
post.title = "New title"
```

runs without error, but after restarting the shell the database still has the old title.

Question:

What was likely forgotten?

```
post.save()
```

Changing the Python object alone does not necessarily persist the change.

67. Day 3 debugging exercise 4

Suppose:

```
Post.objects.get(id=999)
```

raises an error.

Question:

Why?

There is no Post with that id.

This becomes important on Day 4 when we learn how to handle missing records and return a proper 404 page.

68. Day 3 homework

Task 1

Create five blog posts through Django admin.

At least:

```
3 published
2 drafts
```

Task 2

In the Django shell:

```
Post.objects.all()
```

then inspect one record using:

```
post.title
post.content
post.published
post.created_at
```

Task 3

Create a post through the shell using:

```
Post.objects.create(...)
```

Confirm it appears in admin.

Task 4

Update a post in the shell and call:

```
post.save()
```

Confirm the change appears in admin and on the homepage.

Task 5

Explain in your own words:

1. Why do web applications need databases?
 2. What is persistence?
 3. What is a table?
 4. What is a row?
 5. What is a column?
 6. What is a primary key?
 7. What is a Django model?
 8. What is a model instance?
 9. What is an ORM?
 10. What does `makemigrations` do?
 11. What does `migrate` do?
 12. What is Django admin used for?
-

69. Architecture you should be able to draw from memory

By the end of Day 3:

```
Browser
  |
  | HTTP Request
  ↓
URL Configuration
  ↓
View
  |
  | ORM query
  ↓
Django Model
  ↓
ORM
  ↓
Database
  |
  | stored rows
```

```
↓
Model Objects
↓
View
|
| Context
↓
Template
↓
HTML Response
↓
Browser
```

The important new section is:

```
View
↓
Model / ORM
↓
Database
```

70. The main mental model from Day 3

Before Day 3:

```
posts = [...]
```

inside Python.

After Day 3:

```
posts = Post.objects.all()
```

That single change represents a major shift.

Before:

```
temporary hard-coded data
```

After:

```
persistent application data
```

71. Lead into Day 4

At this point, the homepage can display every Post.

But we still have important limitations.

Suppose the database contains:

```
Post 1  
Post 2  
Post 3
```

The homepage shows them all.

Question:

How do you visit only Post 2?

We want routes like:

```
/posts/1/  
/posts/2/  
/posts/3/
```

Question:

How will Django know which record to retrieve?

Using the id from the URL.

Question:

What happens if someone visits:

```
/posts/999/
```

and Post 999 does not exist?

We need a proper 404 response.

That leads naturally into Day 4:

```
QuerySets  
Filtering  
.get()  
Dynamic URLs  
URL parameters  
Primary keys  
Detail views  
get_object_or_404()
```

Day 4 will focus on the **Read** side of CRUD:

Database



Retrieve the correct record



Send it to the correct template



Display one blog post